vasic-digital tmux — Open Issues Tracker

> **Canonical source of truth for everything currently unfinished, partially validated, or at risk of violating the anti-bluff covenant (`Constitution.md` §1 + §11.4.1 through §11.4.6).**
>
> Every PASS in this codebase MUST carry positive evidence captured live that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks. **Tests AND HelixQA Challenges are bound equally.**
>
> Forensic anchor — direct user mandate (verbatim, 2026-04-28 + 2026-05-07 + 2026-05-08, repeatedly reasserted from upstream `vasic-digital` projects):
>
> > "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"
>
> §11.4.6 forensic anchor (verbatim, 2026-05-08):
>
> > "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisely what is happening exactly, in any context, under any conditions, everywhere!"

Compiled: 2026-05-08 (Phase B per-session-containerization cycle). **Author:** Engineering coordinator **Working pool source:** the items below feed the active task list directly. Each item carries a current state, the captured-evidence requirement, and a fix-direction proposal so future-self can resume cold.

> **Migration policy** (mirrors upstream `vasic-digital` projects): resolved items are moved to [**`Fixed.md`**](Fixed.md) in the same commit that closes them. **`Issues.md` holds OPEN / PARTIAL /**

> **BLOCKED / RUNNING / INVESTIGATED only**; once an item is closed and verified end-to-end, it migrates to `Fixed.md` and disappears from here. Never delete items outright — history matters for cold-start handover.

Document conventions

Code	Meaning
`OPEN`	Unfinished work; needs implementation + anti-bluff coverage
`PARTIAL`	Implementation exists but coverage gaps remain (positive evidence missing or environmental SKIP unresolved)
`BLOCKED`	Cannot progress without external dependency (host capability, third-party tool, distro support, etc.)
`RUNNING`	In-flight in this session (background process, ongoing test cycle)
`INVESTIGATED`	Forensic investigation produced findings; closure pending decision on fix scope

Status reclassification rules (§11.4.6 enforced):

- A `PARTIAL` may not move to closed without runtime evidence (`/sys/fs/cgroup/.../memory.max` readback, `systemctl status` showing scope active, `kill -9` survivor proof — never just script exit code).
- A `BLOCKED` reclassifies to `OPEN` when the blocking dependency resolves; it must NOT skip directly to closed.
- An `INVESTIGATED` item must record the captured forensic trace (file path / command output / log timestamp) — never speculation ("likely" / "probably" / "appears to" — see Constitution §11.4.6).

Categories:

- **A** — Tooling / harness gaps
- **B** — Anti-bluff completeness across the existing test surface
- **C** — Per-session containerization features pending evidence
- **D** — Host-capability + topology dispatch gaps
- **E** — Documentation / Continuation drift

A. Tooling / harness gaps

A2 RUNALL-NATIVE-RESOLVE-001 — standalone run_all.sh mis-resolves the binary on native macOS

Status: Fixed (→ Fixed.md) **Type:** Task

CLOSED v1.0.27. `scripts/tests/run_all.sh` hardcodes `TMUX_BIN=tmux/build/bin/tmux` (the `build_containerized.sh` output path) and is the containerized-build validator. On native macOS the authoritative validator is `setup.sh` (builds + verifies against `tmux/build-darwin/`). Invoked standalone on macOS with a stale `tmux/build/` present (a prior Linux containerized build), run_all.sh resolved the wrong-arch binary → `Exec format error` mass-FAIL (observed 2026-06-16; NOT a product defect — `setup.sh` run_all `55/0/6` + installed-binary smoke GREEN the same session; removing the stale `tmux/build/` then gave `not executable` because run_all expects that path). **Fix direction:** make run_all.sh OS-aware (prefer `tmux/build-darwin/` on Darwin) OR document that native-macOS validation is `setup.sh`-only and run_all.sh is the containerized path. Captured-evidence requirement: a clean native-macOS run_all GREEN after the fix.

A3. META-TEST-72-73-COVERAGE-001 — tests 72/73 need persistent meta-test mutations

TMX-ID: TMX-076 **Type:** Task **Status:** Ready for testing

Tests 72 (libevent/ncurses local-dependency obtain) and 73 (build_native.sh local-dependency wiring) currently have no paired §1.1 mutation registered in scripts/tests/meta_test_false_positive_proof.sh, so a regression in either mechanism would not be mechanically caught by the layer-4 anti-bluff sweep. Re-filed follow-up from TMX-071 (originally tracked 2026-06-30 in commit `8232b15`, reverted the same day in commit `9b719a6` purely because of an unrelated add→db-to-md rendering tooling defect, not because the underlying work was invalid or completed — confirmed via git history per §11.4.124; the mutation-writing itself was never done). Acceptance: two new persistent mutations (mirroring the existing M-test67/CM-LOCAL-DEPS-MECHANISM pattern) that mutate a real invariant each test enforces and assert the harness FAILs, then restore cleanly; both wired into the standing sweep alongside the existing 60+ mutations.

A4. NEW-COLLISION-GUARD-SCOPE-GATED-001 — the `new` verb's Linux collision guard is skipped entirely when systemd user-scopes are unavailable

TMX-ID: TMX-077 **Type:** Bug **Status:** Ready for testing

Found by the final whole-branch review of the wizard/password redesign (2026-07-05), confirmed pre-existing (not introduced by that plan) via direct inspection of scripts/tmx.template around the `new` verb's collision check. On Linux, `tmx new -s NAME` refuses to proceed when a session already occupies NAME's systemd scope — but that refusal is gated by `[ "$_scope_ok" -eq 1 ]`, and `_scope_ok` is set to 0 whenever `systemctl --version` reports below 230, `systemctl` is entirely absent, or a real `systemd-run --user --scope` probe fails (see the topology probe a few lines above the guard). On any such host, the guard is skipped outright rather than falling back to an alternate collision check (the way Darwin uses raw socket presence instead), so `tmx new -s NAME` against an already-live session of the same name could proceed further than intended before tmux's own internal duplicate-session refusal (if any) is reached — including possibly reaching the password-collection prompt for what the operator believes is a brand-new session. Not yet reproduced end-to-end on a real scope-unavailable host (this project's dev/CI hosts all have a working systemd user session), so the exact downstream behavior (does tmux's own duplicate-session check catch it first, and if not, could a live session's persisted password state be disturbed) is UNCONFIRMED pending that reproduction. **Fix direction:** give Linux a scope-independent fallback collision check mirroring Darwin's socket-presence check (e.g. `tmux -L SOCK_LABEL has-session -t NAME` before proceeding) so the refusal does not depend on `_scope_ok` at all. **Acceptance:** a test that forces `_scope_ok=0` (or runs on a genuinely scope-less host) and asserts `tmx new -s NAME` against an already-live NAME still refuses cleanly, with no password prompt reached.

B. Anti-bluff completeness across the existing test surface

(Prior B-items closed: B3 P5-M20/P5-M21 escapes CLOSED in v1.0.16 [tests 49/50 + meta-test retarget], state-verified 2026-05-29 with `MUTATIONS CAUGHT 45 / ESCAPED 0`, migrated to `Fixed.md` §B3 as TMX-054; B1 CHAL-COVER-001, B2 TEST-AUDIT-001 also in `Fixed.md`. New open work below.)

C. Per-session containerization features pending evidence

(none open at this time — C1 TMX-T5, C2 TMX-T7, C3 TMX-T8 landed in `Fixed.md`.)

D. Host-capability + topology dispatch gaps

D2 TMPDIR-HARDCODE-001 — tests hardcoding /tmp false-FAIL under host disk-pressure

Status: Fixed (→ Fixed.md) **Type:** Task

CLOSED v1.0.27. Several tests create scratch under a hardcoded `/tmp` (e.g. `27_state_persistence.sh` target `tmx-test-18-target-*`). When the host root volume is full (observed 2026-06-16 on macOS, `/` at <200 MiB during the operator's away-window), the `cd`/mkdir into `/tmp` fails → `pane_current_path=''` false-FAIL instead of an honest §11.4.3 SKIP-with-reason. The tests pass normally + standalone (27 `3/3`, 38 `3/3`, 43 `15/0` re-run the same session) — the failure is purely the abnormal host-disk condition (a §11.4.1 FAIL-bluff class: environment, not product). **Fix direction:** route test scratch through `$TMPDIR` (operator now sets `/Volumes/T7/tmp` via ~/.zshrc + ~/.bashrc) OR guard each test on `/tmp` writability and SKIP-with-reason (§11.4.3/§11.4.50). Captured-evidence requirement: an induced-disk-full run shows SKIP-with-reason, not FAIL.

E. Documentation / Continuation drift

(CONTINUATION.md §3 entries that resolve land in `Fixed.md` per Constitution §5 / §12.10. New open work below.)

F. Runtime crash — operator-gated reproduction

(none open — F1 `tmx` session named "HelixCode" crashes the whole terminal RESOLVED 2026-06-13, operator-confirmed, migrated to `Fixed.md` A45 with reproduced root cause [stale wrapper `TMUX_BIN` pointing at a non-existent prior-checkout path →

`exec` of missing binary → login shell dies → terminal closes] + 4-layer regression guard [test 60 + verify gate `CM-TMX-WRAPPER-TMUXBIN-VALID` + meta `M-WRAPPER-TMUXBIN`]).

Last reviewed: 2026-06-19 (v1.0.27 WIP burn-down. §11.4.138 operator-escape A51 [name:color prompt rejection] FIXED — shell-init + SSH-dispatcher char-set now allows `:` and `#`, max-length 64→80; test 65 operator-escape guard PASS=6/6. M24/A2/D2 in parallel streams per §11.4.103.)

M24-ESCAPE-001 — meta-test M24 (hostname 4-surface color) escapes: test 26 misses a 3-set-line removal

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** Minor (test-coverage gap, no user-facing break — closed v1.0.27: `count=1` removed from M24 regex → strips from BOTH `_apply_color` and `_apply_host_color`, test 26 now FAILs on the mutation → CAUGHT) **Closure:** meta-test 37 CAUGHT / 0 ESCAPED (was 34 CAUGHT / 3 ESCAPED pre-fix)

What: The §1.1 paired-mutation `M24` in `scripts/tests/meta_test_false_positive_proof.sh` removes three of the four `tmux set -g …` lines in `_apply_host_color()` and expects test 26 to FAIL. The harness reports `MUTATION ESCAPED` — test 26 does not FAIL, because it asserts only a subset of the four surfaces, so removing the un-asserted set-lines leaves the test green.

Evidence: `bash scripts/tests/meta_test_false_positive_proof.sh 2>&1 | grep M24` → `FAIL: M24: MUTATION ESCAPED — test 26 did not FAIL with the three set-lines removed`. Confirmed pre-existing (M24 added in commit `f151d13`, v1.0.9 — before the per-session-color feature). Surfaced 2026-06-19 during the per-session-color M25/M26 verification run.

Fix direction: strengthen test 26 to assert ALL FOUR surfaces (`status-style`, `pane-active-border-style`, `clock-mode-colour`, `window-status-current-style`) so the M24 mutation (removing any 3) reliably FAILs it — mirroring the all-4-surfaces assertion already proven in test 63 T3 for the per-session path. (This also closes the symmetry gap: the per-session path has a 4-surface guard; the hostname path should too.)

Out of scope: the per-session-color feature (TMX-051). Tracked separately so the color release is not blocked by an unrelated pre-existing test-gap.

G. Interactive wizard + session-password redesign (2026-07-05)

New OPEN work from the 14-task wizard + session-password redesign plan (`docs/superpowers/plans/2026-07-05-tmx-wizard-password-redesign.md`; spec `docs/superpowers/specs/2026-07-05-tmx-wizard-password-redesign-design.md`). Code lands across sibling tasks of that plan; these four entries track the four user-visible requirements from the operator mandate (random-suffix create, masked password input, single-prompt reopen, existing-session picker).

G1 WIZARD-SUFFIX-001 — wizard-created sessions get a random 4-digit name suffix

TMX-ID: TMX-072 **Type:** Feature **Status:** Implemented (→ Fixed.md)

Typing a session name at the interactive tmx wizard now always creates a brand-new session whose real name is the typed name plus a random 4-digit suffix (e.g. my-session-2507), so retyping the same base name later can never collide with or be confused for an earlier session. This makes every session created through the wizard genuinely unique by construction, while scripts and tests that need a deterministic exact name can set TMX_EXACT_NAME=1 to opt out. Implemented in scripts/tmx-shell-init.sh.template. Acceptance: test 78 passes, showing the created session name matches base-NNNN and that TMX_EXACT_NAME=1 suppresses it.

G2 PASSWORD-MASK-001 — password input is masked with asterisks while typing

TMX-ID: TMX-073 **Type:** Feature **Status:** Implemented (→ Fixed.md)

Session passwords are no longer echoed in plaintext to the terminal while being typed. Every password prompt in the tmx wrapper now shows a single asterisk character for each keystroke, with backspace erasing one asterisk, so a password can never be read off the screen by someone glancing at it. Implemented via the shared _read_password_masked helper in scripts/tmx.template. Acceptance: test 77 passes, proving the pane buffer never contains the typed plaintext.

G3 DOUBLE-PROMPT-001 — reopening a password-protected session no longer asks for the password twice

TMX-ID: TMX-074 **Type:** Bug **Status:** Fixed (→ Fixed.md)

Reopening a session that had been idle-recycled (its tmux process torn down for inactivity, but its password remembered) used to show a confusing second prompt that looked like it might be resetting the password, even though typing the same password both times always worked. The root cause was the attach command checking the remembered password before checking whether the session was actually still running, so a doomed attach attempt fell through to the create flow, which unconditionally asked to set a password again. Opening an already-protected session (live or recycled) now verifies the password exactly once; only a genuinely brand-new session name asks for a password and a confirmation. Fixed in scripts/tmx.template's attach and new command handling. Acceptance: test 81 reproduces the exact reported scenario end-to-end and proves exactly one prompt appears, with the stored password unchanged afterward.

G4 WIZARD-PICKER-001 — wizard offers a picker of existing sessions when no new name is typed

TMX-ID: TMX-075 **Type:** Feature **Status:** Implemented (→ Fixed.md)

Previously, pressing Enter without typing a session name at the interactive tmx wizard always dropped the operator into a plain shell with no other option. Now, if any sessions already exist, the operator sees a numbered list of them plus a 'None' option, and can pick a number to join that session directly (still prompted for its password exactly once if it is protected) instead of having to remember and retype its exact name. Choosing None, or pressing Enter again, behaves exactly as before (a plain shell). Implemented in scripts/tmx-shell-init.sh.template. Acceptance: test 79 passes, covering picking a plain session, picking a password-protected one, and choosing None.

G5 SANITIZE-NAME-001 — session names containing spaces or special characters are normalized to safe names

TMX-ID: TMX-078 **Type:** Feature **Status:** Implemented (→ Fixed.md)

When the operator types or passes a session name containing spaces, tabs, or other special characters, tmx now normalizes it instead of rejecting it: leading/trailing whitespace is removed, internal whitespace runs are collapsed to a single `-`, and any remaining characters outside the session-name safe set are stripped. This applies both to `tmx new -s NAME` (via `scripts/tmx.template`) and to the interactive wizard prompt (via `scripts/tmx-shell-init.sh.template`), so names like `"hello world"` become `hello-world`. The wizard prompt preserves inline colour syntax (`name:red`, `name:#hex`) by splitting at the first `:` before sanitizing the name and inserting the random suffix before the colour token (`home-1234:red`). Empty or whitespace-only input continues to fall through to the existing empty/default picker path. Closure: v1.0.35 / versionCode 36.

H. Pre-existing timing issues surfaced during the 2026-08-10 no-limits-by-default cycle

Discovered while validating TMX-079 (see `Fixed.md`). Confirmed via §11.4.114 A/B isolation against the v1.0.38 baseline (identical failures reproduced BEFORE any of TMX-079's changes were applied) — NOT caused by TMX-079, and TMX-079's own fix + tests are unaffected. Tracked here so they are not lost.

H1 STATE-HOOK-RACE-001 — test 27 sub-check "18" (run-shell cwd-record hook) fails deterministically on iteration 1

TMX-ID: TMX-080 **Type:** Bug **Status:** Queued

What: `scripts/tests/27_state_persistence.sh`'s sub-check labelled "18" drives the wrapper's `run-shell "$STATE_BIN record $SESS #{pane_current_path}"` hook directly (the same command the wrapper installs on `client-detached`/`session-closed`) and polls `tmx-state-bin recall` for up to 5 s waiting for the write to land. Iteration 1 of 3 fails deterministically — `recall` still returns the PRE-hook (Phase-1 target) path, never the hook's own path — both in isolation (3/3 runs) and inside the full suite.

Evidence: `bash scripts/tests/27_state_persistence.sh` → `FAIL 18 iter=1: run-shell hook did not update state — recall='...-target-<pid>' (expected '...-hook-<pid>-1')`, reproduced identically 3× in isolation and 2× on the v1.0.38 baseline (pre-TMX-079) via `git stash` A/B, 2026-08-10.

Discovery note (this is the FIRST time it is tracked as its own item, hence `Queued` not `Reopened` per §11.4.34 — the latter presupposes a prior closure, and this defect was never previously closed): previously only described inline as an accepted "harness timing race" in the test file's own comments (lines ~150-186 of `scripts/tests/27_state_persistence.sh`), which this investigation shows is an INCOMPLETE diagnosis (the failure is NOT purely a full-suite-CPU-contention artifact — it reproduces in isolation too, on an otherwise idle host). Discovered by AI, 2026-08-10, during TMX-079 validation.

Fix direction (not yet investigated to root cause): the test's own comments already correctly rule out a slow `cd`-landing race (T2's separate poll-for-cwd loop) and a `run-shell`-fire vs `record`-write-completion race (the 5 s poll). Iteration 1 specifically failing (not 2 or 3) is CONSISTENT WITH a first-invocation-only effect — e.g. `tmx-state-bin`'s DB/lock-file initialization on a cold state file, or a `run-shell` first-dispatch latency inside a freshly-created tmux server — but this is UNCONFIRMED: no instrumentation has yet isolated the actual cause. Needs a dedicated systematic-debugging pass (§11.4.102) with additional instrumentation (e.g. logging `tmx-state-bin record`'s own start/end timestamps) before a fix is attempted — out of scope for TMX-079.

H2 SPLIT-QUIET-SETTLE-001 — test 87 G4 ("quiet-phase control") occasionally fails to observe a zero-throttle settle window

TMX-ID: TMX-081 **Type:** Bug **Status:** Queued

What: `scripts/tests/87_server_scope_split.sh` G4 waits for a 1 s idle window with `nr_throttled` delta 0 on BOTH the server scope and the workload slice (settling past shell-startup noise) before G5 measures throttling under deliberate load. G4 fails ("quiet-phase never settled") with a small non-zero delta on the workload slice.

Evidence: reproduced identically on both the TMX-079-fixed tree and the v1.0.38 baseline (`git stash` A/B, 3× each, 2026-08-10) — `FAIL: G4: quiet-phase never settled (never-settled: .../tmxw-t87_<pid>.slice:delta=10)`. The isolation invariant G4 exists to set up for (G5: the server scope is NEVER co-throttled while the slice is) PASSES consistently regardless, so the feature under test is proven sound — this is a control-window settle heuristic being slightly too strict/short, not a defect in the isolation mechanism itself.

Discovery note (this is the FIRST time it is tracked as its own item, hence `Queued` not `Reopened` per §11.4.34 — the latter presupposes a prior closure, and this defect was never previously closed): discovered by AI, 2026-08-10, during TMX-079 validation.

Fix direction (not yet investigated to root cause): needs either a longer settle window or a higher delta tolerance for the workload slice specifically (shell/pane-shim startup on a freshly-joined slice may legitimately cost a few throttled periods even at rest) — which of the two (or another cause entirely) is UNCONFIRMED without the actual `cpu.stat` samples across the settle window; needs a dedicated systematic-debugging pass (§11.4.102) before deciding — out of scope for TMX-079.